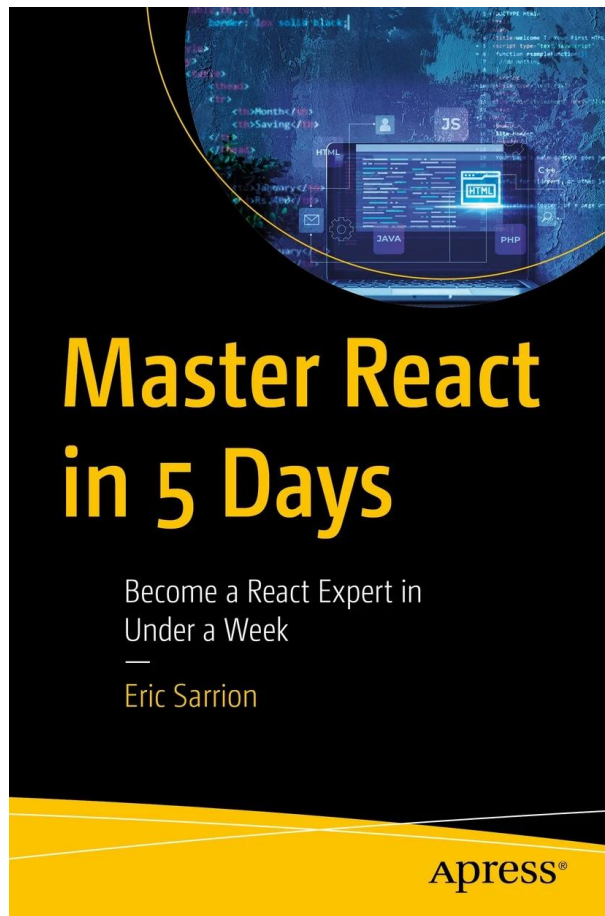


React

Master React in 5 Days



Sarrion, E. (2023). *Master React in 5 Days: Become a React Expert in Under a Week* (1st ed. 2023.). Apress.

ebook beschikbaar op <https://limo.libis.be/VIVES>

<https://doi.org/10.1007/978-1-4842-9855-8>

[hans-naert/reactapp-Master-React-in-5-Days \(github.com\)](https://github.com/hans-naert/reactapp-Master-React-in-5-Days)

1

— **React Components**

2

— **JSX Code**

3

— **Event Handling**

4

— **Hooks**

5

— **Application: Task List**

6

— **Javascript Reminders**



— Hoofdstuk 1: React Components

Inhoud

- Why use React
- React Virtual DOM
- Decomposing an Application into Components
- Analyzing the Main Files of the React Application
- JSX Syntax in React
- Creating a First React Component

— Hoofdstuk 1: React Components

Why Use React

- Wat?
 - Open source library om interactieve user interfaces te maken voor Web Apps
- Waarom?
 - Herbruikbare componenten
 - Virtual DOM: representatie van de UI in het geheugen
 - Documentatie en grote community die meebouwt aan React
 - Multi-platform: Web, mobile, desktop
 - React Native
 - Ecosysteem: Redux, GraphQL, Next.js, Gatsby, Material UI

Hoofdstuk 1: React Components

React Virtual DOM

- Virtual DOM:
 - representatie van de UI in het geheugen (lichte kopie van de echte DOM)
 - DOM aanpassingen traag => componenten passen Virtual DOM aan
 - wijziging status => aanpassing van Virtual DOM
 - Reconciliatie (render):
 - vergelijken van oude Virtual DOM ⇔ nieuwe Virtuale DOM
 - => alle wijzigingen in één keer aan DOM aanbrengen

```
<div>  
<h2> Counter: 0</h2>  
<button>Increment</button>  
</div>
```

```
<div>  
<h2> Counter: 1</h2>  
<button>Increment</button>  
</div>
```

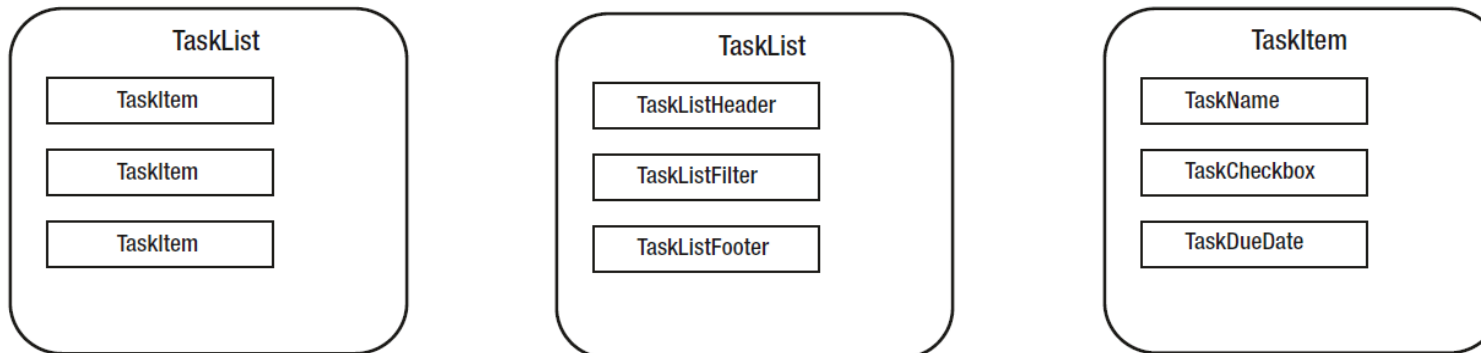


Counter: **1**

Hoofdstuk 1: React Components

Decomposing an Application into Components

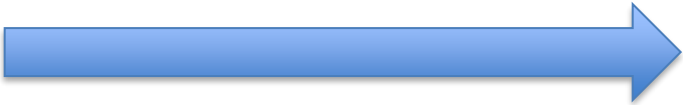
- Het opbouwen van de code met componenten zorgt voor:
 - Begrijpbare code
 - Onderhoudbaarheid (mogelijk om aanpassingen te doen)
 - Herbruikbare componenten
 - Debugbaarheid: elke component apart testbaar

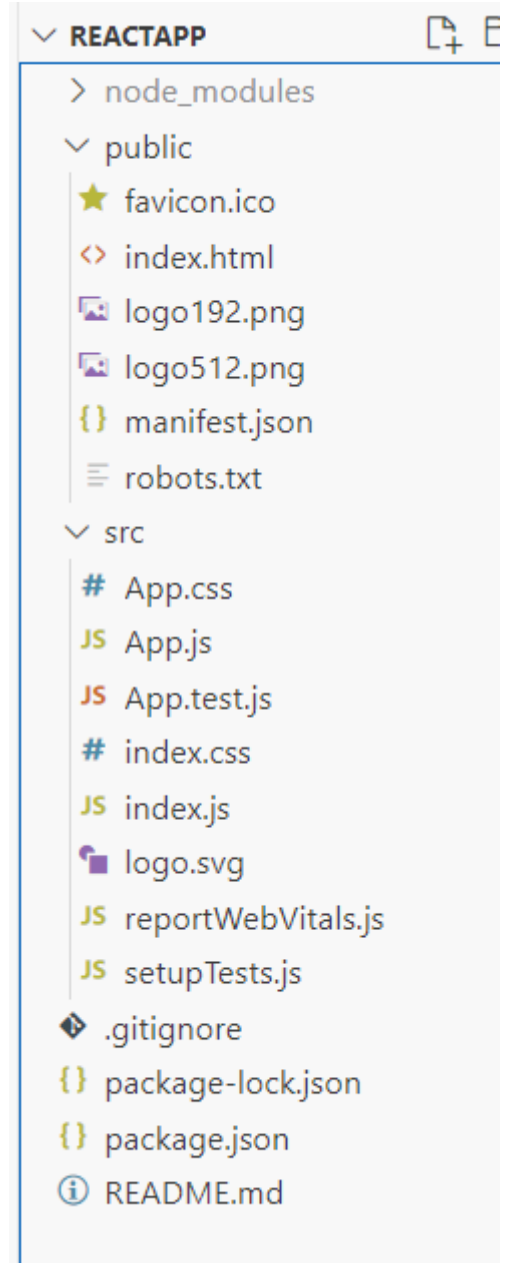


Bron: Sarrion, E. (2023). *Master React in 5 Days*(1st ed. 2023.). Apress.

Hoofdstuk 1: React Components

Analyzing the Main Files of the React Application

- Installatie van Node.js (selecteer alles):
winget install Node.js -i
 - Aanmaak React App ([hans-naert/reactapp \(github.com\)](https://github.com/hans-naert/reactapp)):
npx create-react-app reactapp
 - App structuur
- 
- React frameworks
 - create-react-app => om React te leren
 - Next.js, Remix, Gatsby, Expo => productiekwaliteit
([React-frameworks van productiekwaliteit](#))



Hoofdstuk 1: React Components

Analyzing the Main Files of the React Application

App componenten => render de App component in het root element:

- index.html:

```
31 | | <div id="root"></div>
```

- index.js

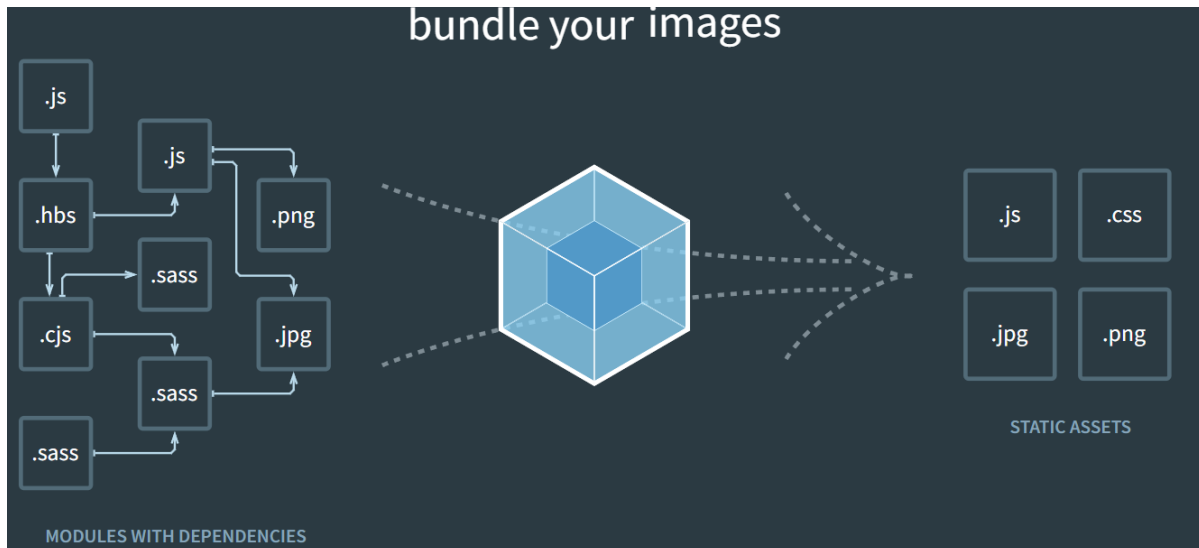
```
7  const root = ReactDOM.createRoot(document.getElementById('root'));
8  root.render(
9    <React.StrictMode>
10   |   <App />
11   |   </React.StrictMode>
12   );
```


Hoofdstuk 1: React Components

Analyzing the Main Files of the React Application

Build: transpilation + bundling

- Transpilation: conversie van nieuwe Javascript/JSX naar Javascript voor alle browsers => [Babel · Babel \(babeljs.io\)](https://babeljs.io/)
- Bundle met "[webpack](https://webpack.js.org/)":



```
src > JS App.js > ...
```

```
1 import logo from './logo.svg';
```

```
2 import './App.css';
```

```
<img src={logo} className="App-logo" alt="logo" />
```

Hoofdstuk 1: React Components

JSX Syntax in React

JSX: JavaScript and XML

- Met JSX, componenten aanmaken zoals in HTML – JSX code wordt omgezet naar een Javascript Object
- Functie App() retourneert een Javascript Object, variabelen tussen accolades {logo}:

```
4  function App() {  
5      return (  
6          <div className="App">  
7              <header className="App-header">  
8                  <img src={logo} className="App-logo" alt="logo" />  
            )  
6      }  
7  }
```

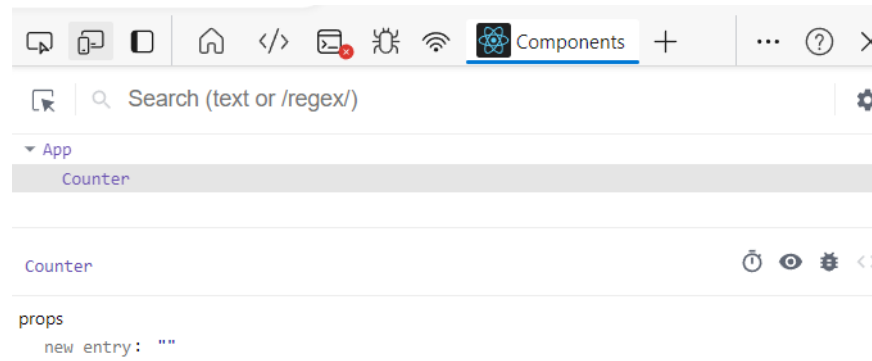
Hoofdstuk 1: React Components

Creating a First React Component

- Counter.js

```
function Counter() {  
  var count = 0;  
  return (  
    <>  
      The counter is set to: {count}  
    </>  
  )  
}  
export default Counter;
```

- React development tools: [React Developer Tools - Microsoft Edge Addons](#)



Hoofdstuk 1: React Components

Creating a First React Component

- Counter incrementeren
 - Code

```
function Counter() {  
  var count = 0;  
  setInterval(function() {  
    count++;  
    console.log("count =", count);  
  }, 1000);  
}
```
 - Variabele wijzigt, maar waarde in browser window blijft 0
=> React weet niet dat de toestand (state) is veranderd en een reconciliatie (render) nodig is.
 - En ... render functie moet "pure" zijn
 - Geen "side effects" => geen aanmaak van timer
 - Deterministisch: zelfde resultaat met zelfde parameters

Hoofdstuk 1: React Components

Creating a First React Component

- Counter incrementeren
 - State

```
import React, { useState } from 'react';
function Counter() {
  const [count, setCount] = useState(0);
  setInterval(function() {
    setCount(count+1);
    console.log("count =", count);
  }, 1000);
```
 - Probleem => setInterval wordt bij elke reconciliation (render) opgeroepen => meerdere timers gelijktijdig actief

Hoofdstuk 1: React Components

Creating a First React Component

- Counter incrementeren
 - useEffect
 - Setup: functie wordt uitgevoerd bij elke reconciliatie (render)
 - Cleanup: Functie die gerouterneerd wordt, deze wordt voor een nieuwe reconciliatie (render) uitgevoerd.

```
useEffect(function() {  
    var timer = setInterval(function() {  
        setCount((count) => (count+1));  
        console.log("count =", count);  
    }, 1000);  
    return function() {  
        clearInterval(timer);  
    }  
});
```

Hoofdstuk 1: React Components

Creating a First React Component

- Counter attributes
 - init attribute meegeven
 - Init attribute via props opvragen

```
<Counter/> <br/>  
<Counter init={5}/> <br/>  
<Counter init="10"/> <br/>
```

```
function Counter(props) {  
  var init = parseInt(props.init || 0);  
  const [count, setCount] = useState(init);
```

— Hoofdstuk 2: JSX code

Inhoud

- Using the `<React.Fragment>`, `<Fragment>` and the `<>` Tag
- Inserting JavaScript Code into JSX
- Writing Conditions in JSX
- Writing a Loop in JSX
- Using Styles in JSX

Hoofdstuk 3: Event handling

Inhoud

- Handle Click Events on a Button
- Managing the Content of an Input Field

— Hoofdstuk 4: Hooks

Inhoud

- Definition of a Hook
- Main Rule About Hooks
- useState()
- useContext()
- useRef()
- useEffect()
- useReducer()
- Other hooks
- Creating Your Own Custom Hook

— **Hoofdstuk 5: Application Task List**

Inhoud

- Application Screens
- Creating the Application with create-react-app
- Breaking Down the Application into Components
- Adding an Item to the List
- Removing an Item from the List
- Modifying an Item in the List

— **Hoofdstuk 6: JavaScript Reminders**

Inhoud

- Using the "let" and "var" Keywords
- Using the "const" Keyword
- Manipulating Objects
- Manipulating Arrays
- Using Import and Export of Modules
- Using Arrow Functions
- Using Promise Objects
- Using "async" and "await" Statements
- Creating an Asynchronous Function That Utilizes JavaScript's "await" Statement